Optical-Channel Subset Sum Recovery and Subspace Folding: A Quantitative Analysis of `run_transparent_bench()`

George Wagenknecht

14 November 2026

Abstract

We present a mathematical and algorithmic analysis of the `run_transparent_bench()` pipeline, which embeds an NP-hard Subset Sum instance into an optical-channel transmission and Chase list-decoding bench. In this architecture, an oracle generates $m =$ rows $\times$ cols integer weights $w \in [10^{11}, 10^{12})^m$ alongside a secret selection bit-vector $b \in \{0,1\}^m$, publishing the target sum $S = w \cdot b$, a 128-bit salt s, and the commitment $c =$ SHA-256$(s \parallel \text{packbits}(b))$. Rather than solving Subset Sum blindly over 2^m combinations, the recogniser observes b across noisy optical exposures via a four-stage subspace fold: (1) 16×16 Kronecker spatial pooling ($256\times$ variance reduction, turning $\sigma = 40.0$ pixel noise into $\sigma_{\text{cell}} = 2.5$ grey levels); (2) ternary confidence gating ($\tau = 127.0$, $c_0 = 25.0$); (3) per-bit sufficient-statistic accumulation $D_i = \text{ones}_i - \text{zeros}_i$ governed by Gumbel cover-time asymptotics and a Chernoff-Bhattacharyya exponent $-\ln(1 - r(1 - Z)) = 0.392$ nats/cycle; and (4) bounded bit-flip Chase decoding ($L = 16, t = 4, V(16,4) = 2,517$ candidates) gated by the dual arithmetic-cryptographic filter $[w \cdot \hat{b} = S] \wedge [\text{SHA-256}(s \parallel \text{packbits}(\hat{b})) = c]$. Crucially, we analyze the scaling transition built into the script: while the diagnostic printouts reference a legacy 56-bit (8×7) instance, the active configuration sets rows $= 800$, cols $= 700$ ($m = 560,000$ bits, $12,800 \times 11,200$ pixel frames). We derive exact cover times, residual error counts, and memory/runtime bottlenecks at $m = 560,000$, demonstrating why a fixed $L = 16, t = 4$ Chase decoder cannot clear residual bit errors within a 49-cycle horizon at $m = 560,000$ without scaling L and t proportionally to $mp_e(T)$.

1 System Architecture and Subset Sum Formulation

The benchmark `run_transparent_bench()` reformulates an instance of the decision/search Subset Sum problem as a noisy optical communication channel paired with an exact arithmetic and cryptographic verifier:

1. **Oracle (Step 1).** Using a seeded pseudorandom generator (seed $= 42$), the oracle draws $m =$ rows $\times$ cols uniformly distributed 64-bit integers $w_i \in [10^{11}, 10^{12})$ and a secret indicator vector $b \in \{0,1\}^m$ (enforcing $\sum_i b_i \geq 1$). It computes the exact dot-product target $S = \sum_{i=0}^{m-1} w_i b_i$, samples a fresh 128-bit salt $s \leftarrow$ `os.urandom(16)`, packs b into $\lceil m/8 \rceil$ bytes, and publishes (w, S, s, c) where $c =$ SHA-256$(s \parallel \text{packbits}(b))$.

2. **Optical Channel (Step 2A–2D).** Across cycles $T \in \{1, \ldots, 49\}$, the oracle renders b as an $R \times C$ binary grid scaled by 255.0, upsamples each cell to a 16×16 pixel block via `np.repeat`, and adds independent zero-mean Gaussian sensor noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = 40.0$. After spatial pooling and confidence gating, the simulation applies a synthetic binary symmetric channel flip mask Bernoulli(ϵ) ($\epsilon = 0.02$) and an erasure reveal mask Bernoulli(r) ($r = 0.45$).

3. **Recogniser and Dual Verifier (Step 2E–2F).** The decoder maintains per-bit counters $(\text{ones}, \text{zeros})$ and a boolean coverage mask `seen`. Once all m bits have been observed at

least once, it forms the maximum-likelihood majority estimate $\hat{b}_i = \mathbb{I}[\text{ones}_i > \text{zeros}_i]$ and executes a bounded Chase list search, accepting a candidate $\hat{b}$ only when both $w \cdot \hat{b} = S$ and SHA-256$(s \parallel \text{packbits}(\hat{b})) = c$ hold.

From a computational complexity standpoint, this pipeline does not constitute a polynomial-time algorithm for arbitrary worst-case Subset Sum instances where only (w, S) are given. Rather, it is an information-assisted recovery protocol: the optical channel leaks roughly $C = r(1 - H_2(\epsilon)) \approx 0.386$ bits of mutual information per variable per exposure, collapsing an exponential 2^m search into a local Hamming-ball refinement around b.

2 Stage-by-Stage Subspace Folding Analysis

Each processing step in the loop discards an orthogonal noise or search subspace while preserving the hidden selection vector b:

2.1 Fold 1: Kronecker Spatial Pooling ($256\times$ Variance Reduction)

Each frame $G \in \mathbb{R}^{(16R)\times(16C)}$ contains $16 \times 16 = 256$ pixels per grid cell. Reshaping G to $(R, 16, C, 16)$ and averaging over axes $(1, 3)$ implements the Kronecker projection $\text{vec}(P) = (A_R \otimes A_C)\text{vec}(G)$ with $a = \frac{1}{16}\mathbf{1}^T$ and $A_k = I_k \otimes a$. Since $aa^T = 1/16$, $256\,\Pi^T\Pi$ is the exact orthogonal projector onto the m-dimensional subspace of block-constant images:

- At $R = 800, C = 700$, each raw frame comprises $12,800 \times 11,200 = 143,360,000$ floating-point pixels (≈ 1.15 GB per frame in `float64`), which spatial pooling collapses onto $m = 560,000$ cell intensities.
- Sensor noise with $\sigma = 40.0$ grey levels is attenuated by $\sqrt{256} = 16$, leaving a pooled cell standard deviation of $\sigma_{\text{cell}} = 40.0/16 = 2.5$ grey levels.

2.2 Fold 2: Confidence Gating ($\tau = 127.0$, $c_0 = 25.0$)

The gate maps each pooled scalar $p_i \in \mathbb{R}$ onto $\{0, 1, -1\}$ using thresholds $\tau + c_0 = 152.0$ and $\tau - c_0 = 102.0$. With nominal black/white levels $\mu_0 = 0$ and $\mu_1 = 255$, the distance from either nominal level to the opposite decision boundary is 152.0 grey levels, or $152.0/2.5 = 60.8\,\sigma_{\text{cell}}$. Under the Gaussian tail $Q(x) = 1 - \Phi(x)$:

$$\epsilon_{\text{Gaussian}} = Q(60.8) \approx 0, \qquad \rho_{\text{Gaussian}} = Q(40.8) - Q(60.8) \approx 0 \tag{1}$$

Spatial pooling renders the Gaussian sensor noise completely negligible. Consequently, the effective channel statistics are governed entirely by the synthetic bit-flip injection ($\epsilon = 0.02$) and reveal mask ($r = 0.45$) applied in Step 2D.

2.3 Fold 3: Sufficient-Statistic Accumulation and Cover-Time Scaling

Recording $(\text{ones}, \text{zeros})$ folds the binary observation history of bit i onto the signed margin $D_i = \text{ones}_i - \text{zeros}_i$, which is proportional to the exact posterior log-odds $\Lambda_i = D_i \ln((1-\epsilon)/\epsilon)$. Two probabilistic laws govern the cycle count T:

1. **Coupon-Collector Cover Time (T_{cov}).** The decoder remains idle until valid_count $== m$. With $\lambda = -\ln(1 - r) = -\ln(0.55) \approx 0.5978$, the cover time follows a Gumbel law with mean $\mathbb{E}[T_{\text{cov}}] \approx (\ln m + \gamma)/\lambda$ and standard deviation $\text{SD}(T_{\text{cov}}) \approx \pi/(\sqrt{6}\lambda) \approx 2.15$ cycles.
2. **Chernoff-Bhattacharyya Bit Error Rate ($p_e(T)$).** Marginalizing over $N_i \sim \text{Bin}(T, r)$, the per-bit majority error probability decays geometrically with Bhattacharyya parameter $Z = 2\sqrt{\epsilon(1-\epsilon)} = 0.280$:

$$p_e(T) \le (1 - r(1 - Z))^T = (0.676)^T, \qquad -\ln(0.676) = 0.392 \text{ nats/cycle}. \tag{2}$$

2.4 Fold 4: Chase List Decoding and Dual Arithmetic-Hash Verification

When $\hat{b} = \mathbb{I}[\text{ones} > \text{zeros}]$ fails verification, `np.argsort(margins)[:16]` selects the $L = 16$ bits with smallest $|D_i|$ and tests all $\sum_{k=1}^{4} \binom{16}{k} = 2,516$ non-zero flip patterns of weight $k \leq 4$ (plus the 0-flip baseline, totaling $V(16, 4) = 2,517$ candidates per cycle).

A notable feature of `run_transparent_bench()` is its two-stage short-circuit verifier:

```
int(np.dot(weights, cand)) == target_sum and sha256(salt + packbits(cand)) == commit
```
(3)

Evaluating the integer dot product first acts as a fast arithmetic pre-filter. Because the weights are uniformly distributed in $[10^{11}, 10^{12})$, an erroneous candidate collides with S with probability $\sim 10^{-12}$, avoiding unnecessary SHA-256 evaluations on rejected candidates while preserving the 2^{-256} cryptographic soundness guarantee on accepted solutions.

3 Regime Comparison: Legacy 56-Bit vs. Scaled 560,000-Bit Configuration

Inspection of the source code reveals a deliberate scale-up in the simulation parameters: while the console print strings retain labels from the original 56-bit instance (`"First 8 of 56"`, `"All 56 Indices"`, `"Theoretical mean 7.7 cycles"`, and `"3.6x10^16 brute-force average"`), lines 13–14 set rows $= 800$, cols $= 700$ yielding $m = 560,000$ variables. Table 1 contrasts the exact analytical behavior across both regimes.

Table 1: Analytical Comparison Between Legacy ($m = 56$) and Active ($m = 560,000$) Script Regimes

Metric / Parameter	**Formula / Code Expression**	**Legacy** (8×7)	**Active** (800×700)
Subset Sum variables m	rows $\times$ cols	56	$560,000$
Brute-force search space	2^m	7.21×10^{16}	$\approx 9.54 \times 10^{168,576}$
Raw frame resolution	$(16$ rows$) \times (16$ cols$)$	128×112 (14.3 kpx)	$12,800 \times 11,200$ (143.4 Mpx)
Raw frame RAM (`float64`)	$8 \cdot (256m)$ bytes	114.7 KB	1.147 GB
Pooled cell noise σ_{cell}	$\sigma/\sqrt{256} = 40.0/16$	2.50 grey levels	2.50 grey levels
Channel capacity per frame	$m \cdot r(1 - H_2(\epsilon))$	21.6 bits/frame	216,160 bits/frame
Mean cover time $\mathbb{E}[T_{\text{cov}}]$	$(\ln m + 0.5772)/0.5978$	7.70 cycles	23.10 cycles
Expected errors at $T = 23$	$\mu(23) \leq m(0.676)^{23}$	6.8×10^{-3} bits	≤ 68.1 bits
Expected errors at $T = 49$	$\mu(49) \leq m(0.676)^{49}$	2.7×10^{-7} bits	$\leq 2.7 \times 10^{-3}$ bits
Pure majority ($P_{\text{err}} \leq 10^{-2}$)	$\ln(m/10^{-2})/0.392$	22.0 cycles	45.5 cycles
Chase search volume $V(16, 4)$	$1 + \sum_{k=1}^{4} \binom{16}{k}$	2,517 candidates	2,517 candidates

4 Scaling Bottlenecks and Engineering Recommendations

Scaling m by four orders of magnitude ($56 \to 560,000$) introduces four concrete operational consequences in the current implementation:

1. **Chase Decoder Under-Sizing at Cover Time.** At $m = 560,000$, full cell coverage occurs around $T_{\text{cov}} \approx 23$ cycles. At $T = 23$, the upper bound on expected bit errors is $\mu = 560,000 \times 0.676^{23} \approx 68$ erroneous bits (with empirical majority errors still numbering in the dozens). Because the Chase search is hardcoded to $L = 16$ bits and $t = 4$ flips, every list-decoding pass between cycles 23 and ~ 38 is guaranteed to fail while spending $2,517$ full-vector dot products. Decoding succeeds only once $T \approx 40$–45 drives $\mu(T) \leq 4$.

2. **64-Bit Integer Overflow in `target_sum`.** In Step 1, `weights` has dtype `np.int64` with values up to 10^{12}, and `secret_b` has dtype `np.uint8`. In NumPy, `np.dot(weights, secret_b)` promotes to `int64` (signed maximum $2^{63} - 1 \approx 9.22 \times 10^{18}$). While $\mathbb{E}[S] \approx 280,000 \times 5.5 \times 10^{11} \approx 1.54 \times 10^{17}$ remains safely below $2^{63} - 1$ for $m = 560,000$, scaling m past $\sim 3 \times 10^7$ would silently overflow `int64` unless cast to Python arbitrary-precision integers or object arrays prior to summation.

3. **Incremental Dot-Product Updates.** Inside the Chase inner loop, `int(np.dot(weights, cand))` recomputes a $560,000$-element dot product from scratch for each of the $2,516$ flip subsets (1.41×10^9 multiply-accumulates per cycle). Precomputing $S_0 = w \cdot \hat{b}$ once per cycle and updating $S_{\text{cand}} = S_0 + \sum_{i \in \text{flip}} (1 - 2\hat{b}_i) w_i$ reduces candidate evaluation from $O(m)$ to $O(k)$ ($k \leq 4$ additions), yielding a $> 10^5\times$ inner-loop speedup.

4. **Dynamic Chase Windowing and Console Formatting.** Replacing the static $L = 16$ window with an adaptive uncertainty set $U = \{i : |D_i| \leq \tau_{\text{margin}}\}$ (triggered only once $\sum_i p_e(N_i, D_i) < 4$) avoids wasted searches immediately after T_{cov}. Additionally, printing all $560,000$ recovered indices in Step 3 generates ~ 35 MB of standard output; truncating the summary table to k sample rows keeps terminal I/O bounded.

5 Conclusion

The `run_transparent_bench()` script demonstrates how parallel spatial pooling, sufficient-statistic accumulation, and Chase list decoding can recover a $560,000$-bit secret vector defining an NP-hard Subset Sum instance from noisy optical frames. Spatial pooling reduces per-frame dimensionality from 1.43×10^8 pixels to 5.6×10^5 cells with a 24.1 dB SNR gain, majority voting suppresses bit-flip noise exponentially at rate 0.676^T, and the dual Subset Sum plus SHA-256 check certifies exact recovery around $T \approx 42$–46 cycles. Incorporating $O(k)$ incremental sum updates and error-gated Chase search eliminates the primary computational bottlenecks at large m.

References

[1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.

[2] H. Chernoff, "A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations," *Annals of Mathematical Statistics*, vol. 23, no. 4, pp. 493–507, 1952.

[3] P. Erdős and A. Rényi, "On a Classical Problem of Probability Theory," *Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei*, vol. 6, pp. 215–220, 1961.

[4] D. Chase, “A Class of Algorithms for Decoding Block Codes With Channel Measurement Information,” *IEEE Transactions on Information Theory*, vol. 18, no. 1, pp. 170–182, 1972.

[5] D. M. Greenberger, M. A. Horne, and A. Zeilinger, “Going Beyond Bell’s Theorem,” in *Bell’s Theorem, Quantum Theory and Conceptions of the Universe*, Kluwer, pp. 69–72, 1989.

[6] D. Gottesman, *Stabilizer Codes and Quantum Error Correction*, PhD thesis, California Institute of Technology, 1997.

[7] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006.

[8] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.